

MACHINE LEARNING · WEEK 1 · LABORATORY CLASS LC 1

Lab 1: Course Tools, Jupyter Notebooks, Datasets and Project Expectations

Module 1: Foundations — Python for Data Work and Introduction to ML · Fall Semester 2026–2027

Item	Details
Course	Machine Learning
Lecturer	Behnam Kiani · Kiani-bme@hotmail.com · office hours by appointment
Linked lecture	L 1. Introduction to Machine Learning: what ML is and examples of use
Duration	One lab session (approx. 4 academic hours: 60 h of labs ÷ 15 weeks); adjust to the timetable
Environment	Google Colab (recommended) or Jupyter Notebook with Python 3.x
Libraries	NumPy, pandas, matplotlib, scikit-learn
Work mode	Individual work; you may discuss with your neighbour, but write your own notebook
Learning outcomes	LO1 (indicators 1.1, 1.2): load, explore and split data in a notebook; train a basic scikit-learn model LO3 (indicator 3.1): write a short, clearly organised notebook
Deliverable	Your completed notebook Lab01_<Surname>_<Name>.ipynb + the Dataset Proposal table (Part F)
Assessment	Counts towards “Weekly Laboratory Tasks & Coding Exercises” (10% of the course grade)
Deadline	As announced in class / on the course MS Teams channel. Late work loses points.

1. Overview and objectives

This first lab gets you ready for the whole semester. You will set up the tools we use every week, learn how Jupyter notebooks work, load and explore your first datasets, train your first machine-learning model, and learn what is expected from you in the midterm mini-project and final capstone project. No previous machine-learning experience is needed.

By the end of this lab you should be able to:

1. Create, run, organise and save a notebook in Google Colab (or Jupyter).
2. Explain the difference between code cells and Markdown cells, and why the order in which cells are executed matters.
3. Import NumPy, pandas, matplotlib and scikit-learn and check their versions.

4. Load a built-in dataset and a CSV file from the web into a pandas DataFrame and inspect it (`head`, `shape`, `info`, `describe`, `isna`).
5. Make a simple, readable black-and-white plot with matplotlib.
6. Run a complete ML program (load → split → train → predict → evaluate) and explain each step.
7. Judge whether a dataset is suitable for a course project using a checklist.
8. Describe the structure, deadlines and assessment criteria of the course projects.

Connection to the lecture

In Lecture 1 we said that machine learning means learning rules from **data + answers**. Today you will see this in practice: in Part E a k-nearest-neighbours classifier learns to recognise iris species from 120 labelled flowers and is then tested on 30 flowers it has never seen — the train/test idea from the lecture.

2. Before you start

✓	Preparation item	Why
☐	A Google account you can log in to	Google Colab and Google Drive use it
☐	A GitHub account (github.com) — free	Project code is submitted and version-controlled with GitHub
☐	Laptop charged, or a university lab PC	All work is done in the browser
☐	Lecture 1 slides open (optional)	Useful for vocabulary: features, label, model, training, test set
☐	Access to the course MS Teams channel	Announcements, deadlines and help

If you prefer to work on your own computer without internet, install the Anaconda distribution, which includes Python, Jupyter and all libraries used in this course, then start Jupyter Notebook from the Anaconda Navigator. Everything in this handout also works in Jupyter; only the Colab-specific parts (Drive, file upload) differ.

Part A — Setting up Google Colab (≈ 20 min)

Google Colaboratory (Colab) is a free, cloud-based Jupyter notebook environment. Nothing needs to be installed, and the main ML libraries are already available.

Steps

1. Open **colab.research.google.com** in your browser and sign in with your Google account.
2. Click **File → New notebook**. A new notebook called `Untitled0.ipynb` opens.
3. Click the title at the top-left and rename it to `Lab01_<Surname>_<Name>.ipynb`, e.g. `Lab01_Karimov_Aziz.ipynb`.
4. Click **Connect** (top-right) to start a runtime — a virtual computer in the cloud that executes your code. A green tick appears when it is connected.

5. In the first code cell type `print("Hello, Machine Learning!")` and press **Shift + Enter**. The output appears below the cell.
6. Click **File → Save a copy in Drive** (or press **Ctrl + S**). Your notebook is stored in the folder *Colab Notebooks* in your Google Drive.
7. Open **Share** and check the options — later, in group projects, you will share notebooks with teammates (view or edit access).

Important: the runtime is temporary

Colab runtimes disconnect after a period of inactivity and are reset after some hours. **Variables and uploaded files disappear**, but your notebook (the code) stays in Drive. After a reconnect, re-run your cells from the top: **Runtime → Run all**.

Task A1. Add a Markdown cell at the very top of your notebook (click + **Text**) with the following header, filled in with your own details:

```
# Lab 1 – Course Tools, Notebooks and Datasets
**Course:** Machine Learning (BSc in AI)
**Student:** <Name Surname>, group <XX>
**Date:** <dd.mm.yyyy>
```

Part B — Working with notebooks (≈ 30 min)

A notebook is a sequence of **cells**. There are two main types:

Cell type	What it contains	How to run / edit
Code cell	Python code; output (text, tables, plots) appears below it	Shift + Enter runs the cell and moves to the next one
Text (Markdown) cell	Formatted explanations: headings, lists, bold, formulas, links	Double-click to edit; Shift + Enter to render

B.1 Useful keyboard shortcuts

Action	Google Colab	Jupyter Notebook
Run cell and move to next	Shift + Enter	Shift + Enter
Run cell and stay	Ctrl + Enter	Ctrl + Enter
Insert code cell above / below	Ctrl + M then A / B	Esc then A / B
Convert cell to text / Markdown	Ctrl + M then M	Esc then M
Convert cell to code	Ctrl + M then Y	Esc then Y
Delete cell	Ctrl + M then D	Esc then D D
Comment / uncomment line	Ctrl + /	Ctrl + /
Show help for a function	Hover, or type <code>len?</code> and run	Shift + Tab, or <code>len?</code>

Action	Google Colab	Jupyter Notebook
Restart and run everything	Runtime → Restart and run all	Kernel → Restart & Run All

B.2 Markdown essentials

You type	You get
# Title, ## Section, ### Subsection	Headings of level 1, 2, 3
bold and <i>*italic*</i>	bold and <i>italic</i>
- item (one per line)	A bullet list
1. step (one per line)	A numbered list
Text wrapped in backticks	Inline code, e.g. <code>df.head()</code>
$y = wx + b$	A formula (LaTeX), e.g. for the linear model
[scikit-learn] (https://scikit-learn.org)	A clickable link

B.3 Execution order matters

The number in square brackets next to a code cell, e.g. [3], shows the **order** in which cells were executed — not their position. Variables live in the runtime’s memory, so a notebook can “work” only because you ran cells in a strange order. Try this:

```
# Cell 1 – run this FIRST
x = 10

# Cell 2 – run this SECOND
print(x * 2)          # prints 20

# Now go back, change Cell 1 to x = 100 but DO NOT run it.
# Run Cell 2 again: it still prints 20, because memory still holds x = 10.
```

Golden rule of notebooks

Before you submit any notebook in this course, click **Runtime → Restart and run all** (Colab) or **Kernel → Restart & Run All** (Jupyter). If it runs from top to bottom without errors, it is reproducible. The capstone rubric explicitly requires a notebook that “runs correctly” and follows a “clean, reproducible pipeline”.

Task B1. Create the two cells above and reproduce the problem. Then use *Restart and run all* and observe the result.

Task B2. Below them, add a Markdown cell of 2–3 sentences explaining in your own words why execution order can cause hidden bugs.

Task B3. Organise your notebook with Markdown headings for each part of this lab: ## Part A, ## Part B, ... ## Part F. Put your code for each part under its heading.

Part C — Library check and Python warm-up (≈ 30 min)

C.1 Check the ML toolkit

```
import sys
import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import sklearn

print("Python      :", sys.version.split()[0])
print("NumPy      :", np.__version__)
print("pandas     :", pd.__version__)
print("matplotlib  :", matplotlib.__version__)
print("scikit-learn:", sklearn.__version__)
```

The short names `np`, `pd` and `plt` are universal conventions — you will see them in every textbook and on every website. Version numbers will differ from computer to computer; that is normal. If an import fails, see the Troubleshooting table in Section 6.

Task C1. Run the cell and write the versions you obtained in a Markdown cell.

C.2 Python warm-up

These short exercises refresh the Python you need for Week 2. Solve each in its own code cell.

```
scores = [72, 85, 90, 64, 78]

# C2: write a function mean(values) using a for-loop (do not use sum() or numpy)
def mean(values):
    total = 0
    for v in values:
        total += v
    return total / len(values)

print(mean(scores))          # expected: 77.8
print(np.mean(scores))       # the NumPy way gives the same result
```

Task	What to do	Hint
C2	Complete and run <code>mean(values)</code> as above; check it against <code>np.mean</code> .	Expected output: 77.8
C3	Use a list comprehension to create <code>passed = the scores ≥ 70</code> .	<code>[s for s in scores if s >= 70]</code>
C4	Create a dictionary <code>student = {"name": ..., "group": ..., "scores": scores}</code> and print a sentence with an f-string, e.g. <i>Aziz (group AI-24) has an average of 77.8</i> .	<code>f"{student['name']} ...";</code> round with <code>:.1f</code>
C5	Convert <code>scores</code> to a NumPy array <code>a</code> and compute <code>a.max()</code> , <code>a.min()</code> , <code>a.std()</code> and <code>a * 1.1</code> .	Notice: arithmetic on the whole array at once — no loop

Part D — Loading and exploring datasets (≈ 50 min)

In machine learning, data usually lives in a **table**: each row is a *sample* and each column a *feature*; one column may be the *label* (target) we want to predict. In Python such a table is a pandas **DataFrame**.

D.1 A built-in dataset: Iris

The Iris dataset contains 150 flowers from three species (setosa, versicolor, virginica), with four measurements each: sepal length, sepal width, petal length and petal width, in centimetres. It ships with scikit-learn, so no download is needed.

```
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame                    # a pandas DataFrame
df["species"] = df["target"].map(dict(enumerate(iris.target_names)))

df.head()                        # first 5 rows
```

```
print(df.shape)                  # (rows, columns)
df.info()                        # column names, types, missing values
df.describe().round(2)           # summary statistics of numeric columns
```

```
df["species"].value_counts()     # how many flowers of each species?
```

Task D1. Run the cells and answer in a Markdown cell:

- How many samples and how many columns does `df` have?
- Which columns are the **features** and which column is the **label**? Is this a regression or a classification problem?
- Are there any missing values? How do you know?
- Which feature has the largest standard deviation? What does that tell you?
- Is the dataset balanced (similar number of samples per class)?

D.2 Your first plot

In this course we use clear, black-and-white-friendly plots: different **marker shapes** rather than colours, and always labelled axes.

```
fig, ax = plt.subplots(figsize=(6, 4))
for name, marker in zip(iris.target_names, ["o", "s", "^"]):
    part = df[df["species"] == name]
    ax.scatter(part["petal length (cm)"], part["petal width (cm)"],
               marker=marker, facecolor="white", edgecolor="black", label=name)
ax.set_xlabel("Petal length (cm)")
ax.set_ylabel("Petal width (cm)")
ax.set_title("Iris flowers: petal measurements")
ax.legend()
plt.show()
```

Task D2. Run the plot. In one or two sentences: which species is easiest to separate from the others, and why? Then change the plot to show **sepal** length vs **sepal** width. Which pair of features separates the species better?

D.3 Loading a CSV file from the web

Most real datasets come as CSV (comma-separated values) files. `pd.read_csv` can read a file from your computer or directly from a URL. The *tips* dataset records 244 restaurant bills.

```
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv"
tips = pd.read_csv(url)

print(tips.shape)          # expected: (244, 7)
print(tips.dtypes)         # numeric vs text (categorical) columns
print(tips.isna().sum())   # missing values per column
tips.head()
```

```
tips.plot.scatter(x="total_bill", y="tip", color="black",
                  title="Tip vs total bill")
plt.show()
```

Task D3. Answer in Markdown: (a) Which columns are numerical and which are categorical? (b) If we wanted to predict `tip`, would that be regression or classification? (c) If we wanted to predict `time` (Lunch/Dinner), which type would it be? (d) What does the scatter plot suggest about the relationship between bill and tip?

D.4 A messier dataset: missing values

Real data is rarely clean. Load the Titanic passenger list and look for missing values — in Week 3 we will learn how to deal with them.

```
titanic = pd.read_csv(
    "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv")
print(titanic.shape)
titanic.isna().sum().sort_values(ascending=False).head()
```

Task D4. Which columns have missing values, and how many? Which column is probably too incomplete to use? Write your answer in Markdown. (Do not fix the data yet — just describe it.)

D.5 Using your own files in Colab

You will need this for your projects. There are two common options:

```
# Option 1 — upload a file from your computer (it disappears when the runtime resets)
from google.colab import files
uploaded = files.upload()          # a file-chooser button appears
my_df = pd.read_csv("my_data.csv") # use the exact file name you uploaded

# Option 2 — mount Google Drive (files persist between sessions)
from google.colab import drive
drive.mount("/content/drive")
my_df = pd.read_csv("/content/drive/MyDrive/ML_course/my_data.csv")
```

In Jupyter on your own computer, simply put the CSV file in the same folder as the notebook and use `pd.read_csv("my_data.csv")`.

Part E — Your first machine-learning model (≈ 40 min)

This is the program shown in the lecture. It follows the ML workflow: **load data** → **split** → **choose a model** → **train** → **predict** → **evaluate**. Type it yourself rather than copying — it helps you remember the pattern.

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)          # 1. data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)  # 2. split
model = KNeighborsClassifier(n_neighbors=5) # 3. model
model.fit(X_train, y_train)                # 4. train
y_pred = model.predict(X_test)             # 5. predict
print("Accuracy:", accuracy_score(y_test, y_pred)) # 6. evaluate

```

Expected output: Accuracy: 1.0 — all 30 test flowers are classified correctly. Remember from the lecture: Iris is small, clean and easy; on real data a perfect score should make you suspicious.

Task E1. Run the program. Print `X_train.shape` and `X_test.shape`. Explain in Markdown what the numbers mean and why we never evaluate on `X_train`.

Task E2 — experiment. The k-NN model predicts the class of a flower from its k nearest neighbours in the training set. Change `random_state` (which training/test split we get) and `n_neighbors` (k) with `test_size=0.3`, and fill in this table in a Markdown cell:

random_state	k = 1	k = 5	k = 50
0			
3			
7			
42			

Then answer: (a) Why does the accuracy change when only `random_state` changes? (b) What happens with a very large k such as 50, and why? (Hint: the training set has only 105 flowers, 35 of each species.)

Tip: use a loop instead of editing the code 12 times:

```

for rs in [0, 3, 7, 42]:
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=rs)
    accs = []
    for k in [1, 5, 50]:
        m = KNeighborsClassifier(n_neighbors=k).fit(X_train, y_train)
        accs.append(round(m.score(X_test, y_test), 3))
    print("random_state", rs, "->", accs)

```

Task E3 — prediction. Use the trained model ($k = 5$, `random_state=42`, `test_size=0.2`) to predict the species of two new flowers:

```

new_flowers = [[5.1, 3.5, 1.4, 0.2],
                [6.7, 3.0, 5.2, 2.3]]
pred = model.predict(new_flowers)
print(pred, iris.target_names[pred])

```

Task E4 — reflection. In 3–4 sentences, identify in this program the Task (T), Experience (E) and Performance measure (P) from Mitchell's definition, and name one limitation of this model or dataset.

Part F — Finding and evaluating datasets (≈ 30 min)

From Week 3 you will work on **IWS 1: dataset preparation and exploratory analysis** for your midterm mini-project, and in Week 2 there is a consultation (IWST 1) on selecting your dataset. Start searching now.

F.1 Where to look

Source	What you find there	Tips
Kaggle — kaggle.com/datasets	Thousands of datasets with example notebooks and discussion	Filter by file type (CSV) and size; check the licence and the “Usability” score
UCI Machine Learning Repository	Classic, well-documented datasets for teaching and research	Each dataset page lists the task type (classification/regression), number of rows and features
scikit-learn built-in datasets	Small ready-to-use datasets (iris, digits, wine, breast cancer, diabetes, California housing)	Good for practice; for projects prefer a dataset you choose yourself
Government / city open-data portals	Energy, transport, environment, education statistics	Often need more cleaning; check that there is a clear target
Google Scholar / IEEE Xplore	Research papers that describe and cite datasets	Useful for the report: how have others used this data?

F.2 Dataset suitability checklist

Question	Good sign	Warning sign
Is the problem clear?	You can state the target y in one sentence	“I just want to analyse this data”
Is there a label column?	A clear numeric (regression) or categorical (classification) target	No obvious target — unless you plan clustering
Is the size manageable?	A few hundred to tens of thousands of rows; 4–30 features	Under ~100 rows, or files of many GB
Is it tabular?	CSV/Excel table: rows = samples, columns = features	Raw images, audio or long free text (beyond this course)
Is it documented?	Column descriptions and units are given	Cryptic column names with no explanation
Is it legal and ethical?	Public licence; anonymised; no personal identifiers	Names, phone or ID numbers; unclear origin
Is it interesting to you?	You can explain why the prediction would be useful	Chosen only because it was the first result

Task F1. Find **two** candidate datasets for your midterm mini-project and fill in the Dataset Proposal table below (in your notebook as a Markdown table, or on paper). Bring it to the Week 2 consultation.

Field	Dataset 1	Dataset 2
Name and link		
Source (Kaggle, UCI, ...) and licence		
Number of rows × columns		
Target variable y		
Task type (regression / classification / clustering)		
3–5 example features		
Missing values or other problems noticed		
Why is this prediction useful?		

Part G — Project expectations

Projects make up **80%** of your course grade, and they start early. Read this section carefully and ask questions now.

G.1 Assessment overview

Component	Weight	When
Weekly laboratory tasks & coding exercises (this lab included)	10%	Every week
Notebook checks & project-progress submissions	10%	Throughout the semester
Midterm mini-project — notebook, report & 5–7 min demonstration	30%	Week 7
Final capstone project — report & code	30%	Week 15
Final presentation & viva	20%	Final exam week
Total	100%	

G.2 Timeline of independent work (IWS) and consultations (IWST)

Week	Milestone
2	IWST 1 — consultation on selecting a dataset for the midterm mini-project
3	IWS 1 starts — dataset preparation and exploratory analysis (notebook due Week 7)
5	IWST 2 — consultation on model training and evaluation
6	IWST 3 — consultation on choosing and tuning a k-NN model
7	IWS 2 — midterm mini-project: notebook report and 5–7 minute demonstration

Week	Milestone
8	IWS 4 — consultation on decision-tree overfitting and depth tuning
9–10	IWS 5–6 — capstone dataset selection and model-comparison approach
12	IWS 3 — build and save a reproducible ML pipeline notebook
13	IWS 7 — review of each group's capstone plan; roles, checklist, report structure
14	Lab support & feedback: model testing, result checking, presentation rehearsal
15 / exam week	IWS 4 — capstone submission (notebook/code, report, presentation); viva in the final exam week

G.3 How the projects are graded

Midterm mini-project criterion	Weight	What “excellent” looks like
Problem understanding	30%	Clearly explains the prediction/clustering task and why it suits ML
Model choice	30%	Selects a clearly appropriate model and explains the choice convincingly
Evaluation	30%	Uses suitable metrics correctly and explains what the results mean
Limitations	10%	Identifies clear, well-reasoned weaknesses of the model or dataset
Capstone criterion		Weight
Problem understanding, model choice & evaluation (compare at least two models)		25%
Workflow & technical implementation (notebook runs; clean, reproducible pipeline)		25%
Report & notebook clarity (headings, figures, tables)		20%
Presentation & viva (within time; answers questions accurately)		15%
Teamwork & shared contribution (each member explains their part)		15%

G.4 Working in groups and using GitHub

- Projects are done in **pairs or small groups**. Form your group early and agree how you will communicate and split the work (data preparation, modelling, report, presentation).
- Every member must be able to **explain their own part** of the work in the viva.
- Create one GitHub repository per project. Commit regularly with meaningful messages — the history shows everyone's contribution.
- Suggested repository structure:

```

ml-midterm-<group>/
├── README.md           # project title, members, dataset source, how to run
├── data/               # the dataset (or a link to it if it is large)
├── notebooks/
│   └── midterm.ipynb   # runs from top to bottom without errors
├── report/
│   └── report.pdf
└── requirements.txt    # e.g. pandas, scikit-learn, matplotlib

```

Academic integrity

Plagiarism, copying other students' notebooks or submitting code you cannot explain are not acceptable. You may use documentation, textbooks and online examples, but **cite** any dataset, library tutorial or code you adapt (a comment or Markdown note with the link is enough). If you use an AI assistant, you remain responsible for every line and must be able to explain it in the viva. The university's Academic Policy and the Policy of Academic Integrity apply.

3. Deliverables and submission checklist

✓	Check before submitting
☐	Notebook named Lab01_<Surname>_<Name>.ipynb with the header cell from Task A1
☐	Markdown headings for Parts A–F; code under the correct heading
☐	All tasks B1–B3, C1–C5, D1–D4, E1–E4 and F1 completed; written answers in Markdown cells
☐	The E2 experiment table filled in and both questions answered
☐	Dataset Proposal table (F1) with two candidate datasets
☐	Runtime → Restart and run all completes with no errors
☐	Submitted via the channel announced by the lecturer (MS Teams / shared folder / GitHub), before the deadline

4. Marking scheme for Lab 1

Criterion	Excellent (100%)	Good (75%)	Satisfactory (50%)	Unsatisfactory (0–25%)	Weight
Setup & notebook organisation (A, B)	Header, clear headings, runs top-to-bottom	Organised; minor issues	Runs, but disorganised	Missing or does not run	20%
Python & library tasks (C)	All correct and tidy	Mostly correct	Partly done	Not done	15%
Data exploration (D)	All code runs; insightful answers	Code runs; reasonable answers	Some answers missing or vague	Little or no exploration	25%

Criterion	Excellent (100%)	Good (75%)	Satisfactory (50%)	Unsatisfactory (0–25%)	Weight
First model & experiment (E)	Table complete; clear, correct explanations of split, k and T-E-P	Complete; explanations mostly correct	Partly complete	Not done	25%
Dataset proposal (F)	Two well-chosen datasets, all fields filled	Two datasets, minor gaps	One dataset or many gaps	Missing	15%

5. Reflection questions (answer at the end of your notebook)

1. In your own words: what is the difference between traditional programming and machine learning?
2. What is the difference between a feature and a label? Give one example of each from the tips dataset.
3. Why do we split the data into training and test sets? What could go wrong if we did not?
4. Which part of today's lab was the most difficult for you, and what will you do to prepare for Week 2 (pandas basics)?

6. Troubleshooting

Problem	Likely cause	Fix
<code>NameError: name 'df' is not defined</code>	The cell that creates <code>df</code> was not run (e.g. after a reconnect)	Run the notebook from the top: Runtime → Run all
<code>ModuleNotFoundError: No module named ...</code>	Library not installed (local Jupyter)	Run <code>!pip install <package></code> in a cell, or use Anaconda / Colab
<code>FileNotFoundError</code> in <code>read_csv</code>	Wrong file name or folder; file lost after runtime reset	Check spelling with <code>!ls</code> ; re-upload or use Google Drive
<code>HTTPError</code> / URL error when reading CSV	No internet, or a typing error in the URL	Check the connection; copy the URL exactly
Plot does not appear	Missing <code>plt.show()</code> or an error earlier in the cell	Add <code>plt.show()</code> ; read the error message from the top
Different accuracy from your neighbour	Different <code>random_state</code> or <code>test_size</code>	Use exactly the values given in the task
Colab says “Runtime disconnected”	Inactivity or time limit	Reconnect and Run all; your code is saved in Drive
<code>IndentationError</code>	Mixed or missing indentation after <code>for</code> , <code>def</code> , <code>if</code>	Use 4 spaces consistently; Colab auto-indent

7. Further resources

- Google Colab welcome notebook: colab.research.google.com (File → Open notebook → Examples)

- scikit-learn documentation and tutorials: scikit-learn.org (see “Getting Started” and the User Guide)
- pandas “10 minutes to pandas” guide in the pandas documentation — good preparation for Week 2
- Kaggle Learn free micro-courses: “Python” and “Pandas” (kaggle.com/learn)
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.), Ch. 1–2
- Müller, A. C., & Guido, S. (2016). *Introduction to Machine Learning with Python*, Ch. 1

Appendix — Instructor notes and expected results

For the lecturer / lab assistant. Remove this page before distributing the handout if you prefer students not to see the expected values.

Task	Expected result / key points
B1	Cell 2 still prints 20 after editing Cell 1 without running it; after Restart and run all it prints 200.
C2–C5	mean = 77.8; passed = [72, 85, 90, 78]; a.max() = 90, a.min() = 64, a.std() ≈ 9.22; a * 1.1 multiplies every element.
D1	df.shape = (150, 6) after adding the species column (150, 5 before). Features: the four measurements; label: target/species → classification. No missing values (150 non-null in every column). Balanced: 50 per species. Petal length has the largest std (≈ 1.77 cm).
D2	Setosa is clearly separated by petal size. Petal features separate the species much better than sepal features.
D3	tips.shape = (244, 7). Numerical: total_bill, tip, size; categorical: sex, smoker, day, time. Predicting tip → regression; predicting time → classification. Positive, roughly linear relationship.
D4	titanic.shape = (891, 15). Missing: deck 688, age 177, embarked 2, embark_town 2. deck is mostly empty and probably unusable.
E1	X_train (120, 4), X_test (30, 4); accuracy 1.0.
E2 (test_size = 0.3)	random_state 0: 0.978 / 0.978 / 0.911 · 3: 0.956 / 0.956 / 0.911 · 7: 0.933 / 0.911 / 0.822 · 42: 1.0 / 1.0 / 0.956 (for k = 1 / 5 / 50). Accuracy depends on which flowers land in the test set (small test set → high variance). k = 50 is larger than the 35 training flowers per class, so neighbours from other species outvote the correct one → underfitting.
E3	Predictions: [0 2] → setosa, virginica.
E4	T: classify iris species; E: 120 labelled flowers; P: accuracy on the 30 test flowers. Limitations: tiny, clean dataset; a single split; only four features; results may not transfer to real, noisy data.